

```
# libraries to analyze data
import numpy as np
import pandas as pd
import scipy.stats as sps

# libraries to visualize data
import matplotlib.pyplot as plt
import seaborn as sns

# Misc libraries
import random
```

✓ Data loading and exploratory data analysis

```
# read the file into a pandas dataframe
df = pd.read_csv('/content/sample_data/delhivery_data.csv')
# look at the datatypes of the columns
print('*****')
print(df.info())
print('*****\n')
print('*****')
print(f'Shape of the dataset is {df.shape}')
print('*****\n')
print('*****')
print(f'Number of nan/null values in each column: \n{df.isna().sum()}')
print('*****\n')
print('*****')
print(f'Number of unique values in each column: \n{df.nunique()}')
print('*****\n')
print('*****')
print(f'Duplicate entries: \n{df.duplicated().value_counts()}')
print('*****')
```

```
dtype: int64
*****

*****
Number of unique values in each column:
data                2
trip_creation_time   918
route_schedule_uuid 625
route_type          2
trip_uuid           918
source_center       729
source_name         725
destination_center   730
destination_name     726
od_start_time       1650
od_end_time         1650
start_scan_to_end_scan 560
is_cutoff           2
cutoff_factor       289
cutoff_timestamp    7755
actual_distance_to_destination 8174
actual_time         1579
osrm_time           1032
osrm_distance       8086
factor             4682
segment_actual_time  231
segment_osrm_time    99
segment_osrm_distance 7979
segment_factor      1430
dtype: int64
*****

*****
Duplicate entries:
False      8194
Name: count, dtype: int64
*****
```

```
# look at the top 3 rows
df.head(3)
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name	od_start_time	...
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	trip- 153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	...
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	trip- 153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	...
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	trip- 153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	...
3 rows x 24 columns											

Insight:

- A quick look at the information of the data reveals that there are 144867 rows and 24 columns implying 144867 trips have been made with each trip having information such as trip_creation_time, trip_uuid, source_center, source_name, destination_center, destination_name to name a few. Most of the datatype are either "object" or "float64" except for is_cutoff and cutoff_factor.
- We can also infer that there are 293 missing values or null value in source_name and 261 missing values or null value in destination_name in the dataset. As these numbers are small compared to dataset size, 144867, it is safe to drop the rows with the missing values
- There are no duplicate entries.
- As columns is_cutoff, cutoff_factor, cutoff_timestamp, factor and segment_factor are Unknown fields, there is no harm in dropping these columns.
- It makes sense to convert columns data and route_type to "category" datatype
- It makes sense to convert columns trip_creation_time, od_start_time, od_end_time to "datetime" datatype

```
df = df.dropna(how='any')
df = df.drop(columns = ["is_cutoff", "cutoff_factor", "cutoff_timestamp", "factor", "segment_factor"])
df["data"] = df["data"].astype("category")
df["route_type"] = df["route_type"].astype("category")
df["trip_creation_time"] = pd.to_datetime(df["trip_creation_time"], format='%Y-%m-%d %H:%M:%S.%f')
df["od_start_time"] = pd.to_datetime(df["od_start_time"], format='%Y-%m-%d %H:%M:%S.%f')
df["od_end_time"] = pd.to_datetime(df["od_end_time"], format='%Y-%m-%d %H:%M:%S.%f')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 8158 entries, 0 to 8192
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0    data                                8158 non-null   category
1    trip_creation_time                  8158 non-null   datetime64[ns]
2    route_schedule_uuid                8158 non-null   object
3    route_type                          8158 non-null   category
4    trip_uuid                          8158 non-null   object
5    source_center                      8158 non-null   object
6    source_name                        8158 non-null   object
7    destination_center                 8158 non-null   object
8    destination_name                   8158 non-null   object
9    od_start_time                      8158 non-null   datetime64[ns]
10   od_end_time                        8158 non-null   datetime64[ns]
11   start_scan_to_end_scan             8158 non-null   float64
12   actual_distance_to_destination      8158 non-null   float64
13   actual_time                        8158 non-null   float64
14   osrm_time                          8158 non-null   float64
15   osrm_distance                      8158 non-null   float64
16   segment_actual_time                8158 non-null   float64
17   segment_osrm_time                  8158 non-null   float64
18   segment_osrm_distance              8158 non-null   float64
dtypes: category(2), datetime64[ns](3), float64(8), object(6)
memory usage: 1.1+ MB
```

```
df.describe()
```

	trip_creation_time	od_start_time	od_end_time	start_scan_to_end_scan	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	segment_actual_time
count	8158	8158	8158	8158.000000	8158.000000	8158.000000	8158.000000	8158.000000	8158.000000
mean	2018-09-22 23:12:33.327985664	2018-09-23 03:20:51.278299648	2018-09-23 16:58:20.469313280	816.995342	199.789186	355.878647	184.807796	244.137313	35.7543
min	2018-09-12 00:25:19.499696	2018-09-12 00:46:48.079257	2018-09-12 01:53:32.471405	25.000000	9.000267	9.000000	6.000000	9.202000	-26.0000
25%	2018-09-17 06:05:38.430885120	2018-09-17 11:08:13.273317120	2018-09-18 01:19:19.879985920	142.000000	22.674766	47.000000	25.000000	27.904750	19.0000
50%	2018-09-23 01:22:00.932464896	2018-09-23 05:18:58.024569088	2018-09-23 22:46:22.215742976	350.000000	45.884425	107.000000	54.000000	61.635600	28.0000
75%	2018-09-28	2018-09-28	2018-09-29	1087.000000	100.000000	278.000000	186.000000	242.010000	40.0000

Insight:

- The data is provided from 2018-09-12 00:00:16.535741 to 2018-10-03 23:59:42.701692
- The average time taken to deliver from source to destination is 964 mins with least time being 20mins and maximum time being 7898 mins
- The average distance between source and destination warehouse is 235 Kms with least distance being 9 Kms and maximum distance being 1927 Kms

▼ Detailed Analysis

Detecting outliers

Outliers for every continuous variable

```
# helper function to detect outliers
def detectOutliers(df):
    q1 = df.quantile(0.25)
    q3 = df.quantile(0.75)
    iqr = q3-q1
    lower_outliers = df[df<(q1-1.5*iqr)]
    higher_outliers = df[df>(q3+1.5*iqr)]
    return lower_outliers, higher_outliers
```

```
numerical_columns = ['start_scan_to_end_scan', 'actual_distance_to_destination', 'actual_time', 'osrm_time', 'osrm_distance', 'segment_actual_time', 'segment_osrm_time',
column_outlier_dictionary = {}
for column in numerical_columns:
    print('*'*50)
    print(f'Outliers of \'{column}\'' column are:')
    lower_outliers, higher_outliers = detectOutliers(df[column])
    print("Lower outliers:\n", lower_outliers)
```

```
print("Higher outliers:\n", higher_outliers)
print('*'*50, end="\n")
column_outlier_dictionary[column] = [lower_outliers, higher_outliers]
```

```
*****
```

```
Outliers of 'actual_time' column are:
```

```
Lower outliers:
```

```
Series([], Name: actual_time, dtype: float64)
```

```
Higher outliers:
```

```
397      876.0
```

```
398      899.0
```

```
399      956.0
```

```
400      989.0
```

```
401     1038.0
```

```
...
```

```
8132     1130.0
```

```
8133     1151.0
```

```
8134     1193.0
```

```
8135     1236.0
```

```
8136     1248.0
```

```
Name: actual_time, Length: 1089, dtype: float64
```

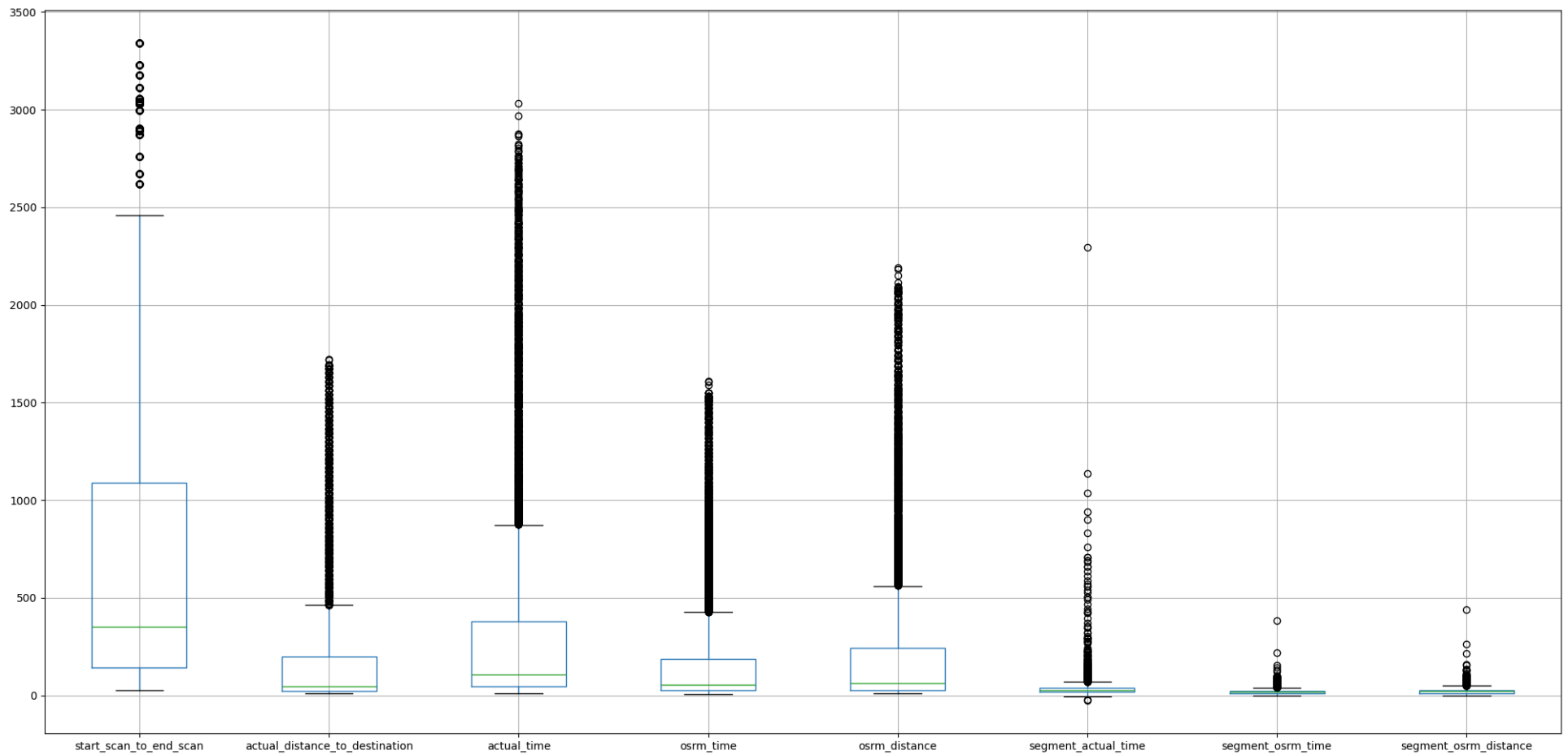
```
*****
```

```
*****
```

```
Outliers of 'osrm_time' column are:
```

```
Lower outliers:
```

```
df[numerical_columns].boxplot(figsize=(25,12))  
plt.show()
```



```
for key, value in column_outlier_dictionary.items():  
    print(f'The column \'{key}\'' has {len(value[0])} + {len(value[1])} outliers')
```

```
The column 'start_scan_to_end_scan' has 939 outliers  
The column 'actual_distance_to_destination' has 1162 outliers  
The column 'actual_time' has 1089 outliers  
The column 'osrm_time' has 1140 outliers  
The column 'osrm_distance' has 1148 outliers  
The column 'segment_actual_time' has 485 outliers  
The column 'segment_osrm_time' has 303 outliers  
The column 'segment_osrm_distance' has 224 outliers
```

Remove the outliers

```
remove_outliers = False  
if True == remove_outliers:  
    for key, value in column_outlier_dictionary.items():  
        lower_outliers = value[0]  
        higher_outliers = value[1]  
        df.drop(lower_outliers.index, inplace=True)  
        df.drop(higher_outliers.index, inplace=True)  
else:  
    print('Not removing any outliers')
```

```
Not removing any outliers
```

Univariate analysis

Numerical Variables

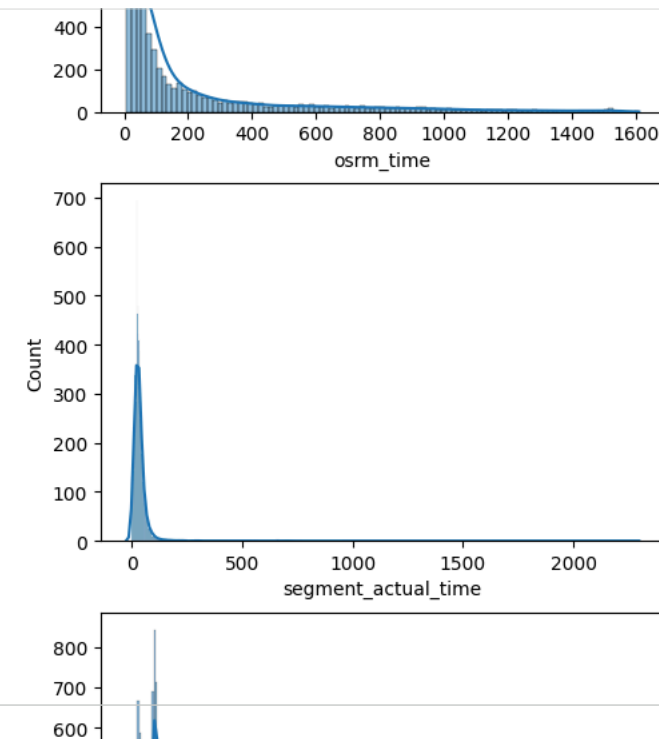
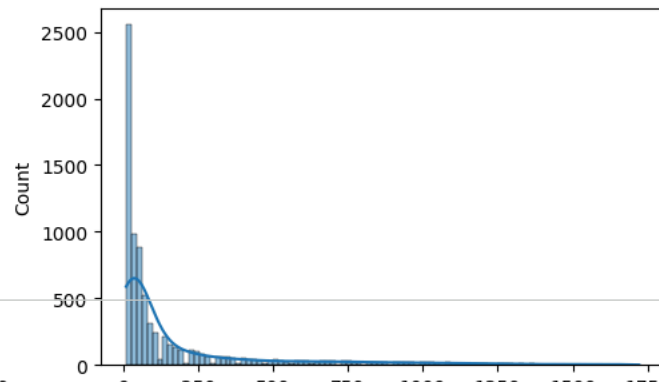
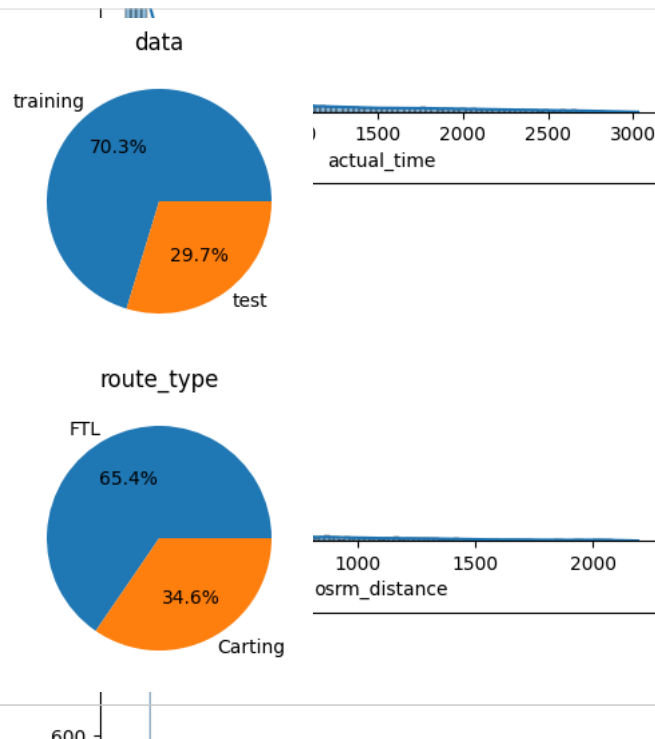
```
fig, ax = plt.subplots(nrows=4, ncols=2, figsize = (12, 16))  
sns.histplot(data=df, x = "start_scan_to_end_scan", kde=True, ax=ax[0,0])  
sns.histplot(data=df, x = "actual_distance_to_destination", kde=True, ax=ax[0,1])  
sns.histplot(data=df, x = "actual_time", kde=True, ax=ax[1,0])  
sns.histplot(data=df, x = "osrm_time", kde=True, ax=ax[1,1])  
sns.histplot(data=df, x = "osrm_distance", kde=True, ax=ax[2,0])  
sns.histplot(data=df, x = "segment_actual_time", kde=True, ax=ax[2,1])  
sns.histplot(data=df, x = "segment_osrm_time", kde=True, ax=ax[3,0])  
sns.histplot(data=df, x = "segment_osrm_distance", kde=True, ax=ax[3,1])  
plt.show()
```


✓ Categorical Variables

```

categorical_columns = ["data", "route_type"]
plt.figure(figsize=(6,6))
plt.subplot(2,1,1)
data = df["data"].value_counts()
plt.pie(data.values, labels = data.index, autopct='%0.1f%%')
plt.title("data")
plt.subplot(2,1,2)
data = df["route_type"].value_counts()
plt.pie(data.values, labels = data.index, autopct='%0.1f%%')
plt.title("route_type")
plt.show()

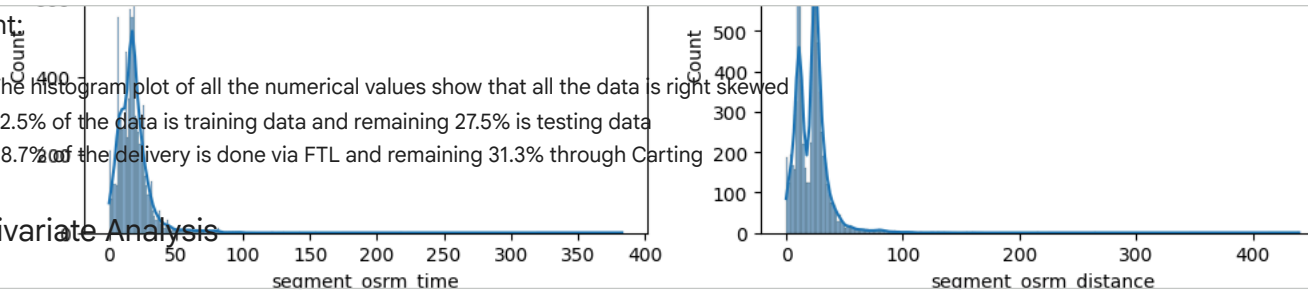
```



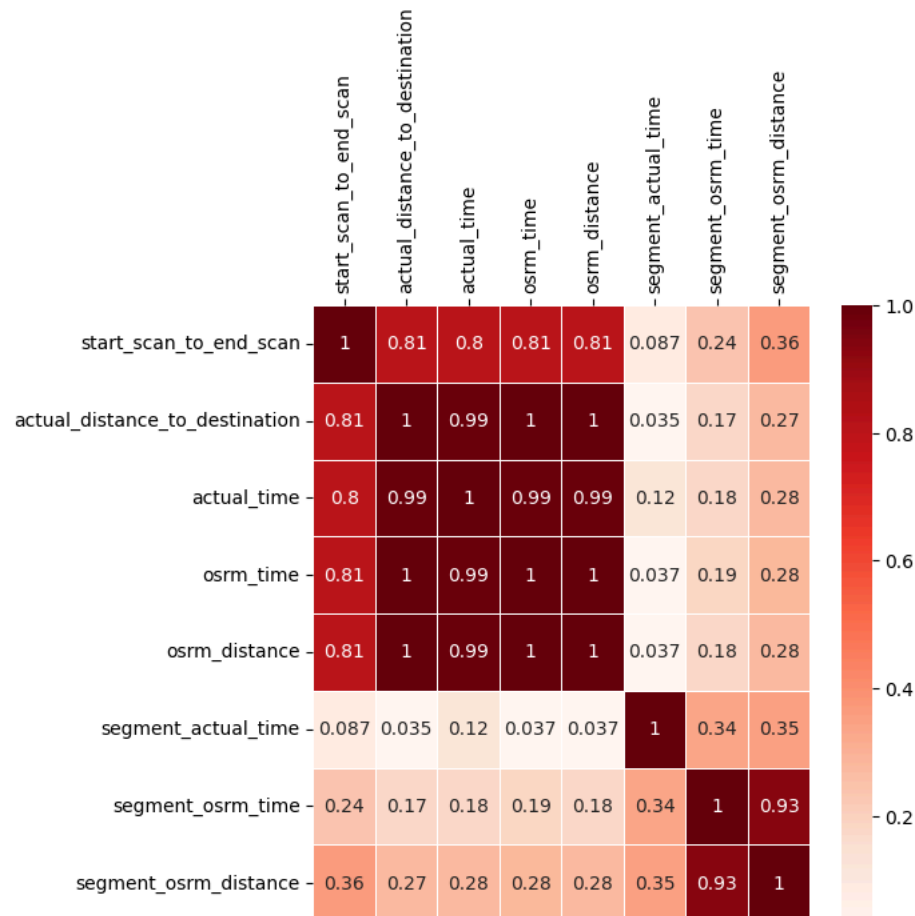
Insight:

- The histogram plot of all the numerical values show that all the data is right skewed
- 72.5% of the data is training data and remaining 27.5% is testing data
- 68.7% of the delivery is done via FTL and remaining 31.3% through Carting

Multivariate Analysis



```
fig, ax = plt.subplots(figsize=(6,6))
sns.heatmap(df.select_dtypes(include=np.number).corr(), annot=True, linewidth=0.5, cmap = "Reds", ax=ax)
ax.xaxis.tick_top()
plt.xticks(rotation=90)
plt.show()
```



Insight:

- The heatmap clearly shows high correlation between time and distance. This is expected as the delivery time increases with increase in distance
- actual_distance_to_destination, actual_time, osrm_time and osrm_distance are highly correlated
- segment_osrm_time and segment_osrm_distance are highly correlated

Merging rows

The delivery details of one package is divided into several rows. Creating a unique identifier, called segment_key, for different segments of a trip based on the combination of trip_uuid, source_center, and destination_center and then merge the rows of columns segment_actual_time, segment_osrm_distance and segment_osrm_time with same segment_key to form new columns segment_actual_time_sum, segment_osrm_distance_sum, segment_osrm_time_sum

```
df["segment_key"] = df["trip_uuid"] + '_' + df["source_center"] + '_' + df["destination_center"]
df = df.drop(columns=["source_center", "destination_center"])
segment_columns = ["segment_actual_time", "segment_osrm_distance", "segment_osrm_time"]
for col in segment_columns:
    df[col + "_sum"] = df.groupby("segment_key")[col].cumsum()
```

df.head(4)

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_name	destination_name	od_start_time	od_end_time	start_scan_to_end_scan
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	Anand_VUNagar_DC (Gujarat)	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797	86.0
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	Anand_VUNagar_DC (Gujarat)	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797	86.0
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	Anand_VUNagar_DC (Gujarat)	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797	86.0
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78- b351-4c0e-a951- fa3d5c3...	Carting	153741093647649320	Anand_VUNagar_DC (Gujarat)	Khambhat_MotvdDPP_D (Gujarat)	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797	86.0

4 rows × 21 columns

Grouping the data by segment_key, with aggregation defined for each column, and creating a new dataframe segment

```
segment_dict = {
    'data': 'first',
    'trip_creation_time': 'first',
    'route_schedule_uuid': 'first',
    'route_type': 'first',
    'trip_uuid': 'first',
```

```
'source_name' : 'first',
'destination_name' : 'last',
'od_start_time' : 'first',
'od_end_time' : 'last',
'start_scan_to_end_scan' : 'first',
'actual_distance_to_destination' : 'last',
'actual_time' : 'last',
'osrm_time' : 'last',
'osrm_distance' : 'last',
'segment_actual_time_sum' : 'last',
'segment_osrm_distance_sum' : 'last',
'segment_osrm_time_sum' : 'last',
}

segment_df = df.groupby('segment_key').agg(segment_dict).reset_index()
segment_df = segment_df.sort_values(by=['segment_key', 'od_end_time'], ascending=True).reset_index()
```

segment_df.head()

index		segment_key	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_name	destination_name
0	0	trip-153671191949943656_IND487001AAB_IND487551AAA	training	2018-09-12 00:25:19.499696	thanos::sroute:0ac760f3-96cb-4046-bfd0-8bc4678...	FTL	trip-153671191949943656	Narsinghpur_KndliDPP_D (Madhya Pradesh)	Gadarwara_MPward_D (Madhya Pradesh)
1	1	trip-153671191949943656_IND487551AAA_IND464668AAA	training	2018-09-12 00:25:19.499696	thanos::sroute:0ac760f3-96cb-4046-bfd0-8bc4678...	FTL	trip-153671191949943656	Gadarwara_MPward_D (Madhya Pradesh)	Bareli_SourvDPP_D (Madhya Pradesh)
2	2	trip-153671237597058150_IND785690AAB_IND785682AAA	training	2018-09-12 00:32:55.970840	thanos::sroute:db0f8027-8ade-4411-9aff-b26adaa...	Carting	trip-153671237597058150	Sonari_Central_DPP_1 (Assam)	Sivasagar_Babupaty_D (Assam)
3	3	trip-153671262893947351_IND500055AAC_IND501401AAC	training	2018-09-12 00:37:08.939733	thanos::sroute:beb73e7f-71ff-4501-bc17-191b4f3...	Carting	trip-153671262893947351	Hyderabad_North_D_2 (Telangana)	Medchal_MROoffice_D (Telangana)
4	4	trip-153671262893947351_IND501401AAC_IND500010AAA	training	2018-09-12 00:37:08.939733	thanos::sroute:beb73e7f-71ff-4501-bc17-191b4f3...	Carting	trip-153671262893947351	Medchal_MROoffice_D (Telangana)	Hyderabad_Alwal_I (Telangana)

Next steps: [New interactive sheet](#)

Feature Engineering

Extracting features from given data

Extracting time taken between od_start_time and od_end_time

```
segment_df['od_time_diff_hour'] = (segment_df['od_end_time'] - segment_df['od_start_time']).dt.total_seconds()/3600
segment_df = segment_df.drop(columns=['od_end_time', 'od_start_time'])
```

segment_df.head(4)

	index	segment_key	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_name	destination_name
0	0	trip-153671191949943656_IND487001AAB_IND487551AAA	training	2018-09-12 00:25:19.499696	thanos::sroute:0ac760f3-96cb-4046-bfd0-8bc4678...	FTL	trip-153671191949943656	Narsinghpur_KndliDPP_D (Madhya Pradesh)	Gadarwara_MPward_D (Madhya Pradesh)
1	1	trip-153671191949943656_IND487551AAA_IND464668AAA	training	2018-09-12 00:25:19.499696	thanos::sroute:0ac760f3-96cb-4046-bfd0-8bc4678...	FTL	trip-153671191949943656	Gadarwara_MPward_D (Madhya Pradesh)	Bareli_SourvDPP_D (Madhya Pradesh)
2	2	trip-153671237597058150_IND785690AAB_IND785682AAA	training	2018-09-12 00:32:55.970840	thanos::sroute:db0f8027-8ade-4411-9aff-b26adaa...	Carting	trip-153671237597058150	Sonari_Central_DPP_1 (Assam)	Sivasagar_Babupaty_D (Assam)
3	3	trip-153671262893947351_IND500055AAC_IND501401AAC	training	2018-09-12 00:37:08.939733	thanos::sroute:beb73e7f-71ff-4501-bc17-191b4f3...	Carting	trip-153671262893947351	Hyderabad_North_D_2 (Telangana)	Medchal_MROoffice_D (Telangana)

Next steps: [New interactive sheet](#)

Extracting city, place, code and state from source_name and destination_name

```
segment_df['source_state'] = segment_df['source_name'].str.extract(r'\((.*?)\)')
segment_df['source_data'] = segment_df['source_name'].str.extract(r'^(.*?)\(')
segment_df['source_data'] = segment_df['source_data'].str.strip()

segment_df['destination_state'] = segment_df['destination_name'].str.extract(r'\((.*?)\)')
segment_df['destination_data'] = segment_df['destination_name'].str.extract(r'^(.*?)\(')
segment_df['destination_data'] =segment_df['destination_data'].str.strip()
```

```
def extract_city_place_code(name):
    parts = name.split('_')
    num_of_parts = len(parts)
    if(num_of_parts == 3):
        city = parts[0]
        place = parts[1]
        code = parts[2]
    elif(num_of_parts == 2):
        city = parts[0]
        place = parts[1]
        code = 'none'
    else:
        city = parts[0]
        place = city
        code = 'none'

    if city == 'Bangalore' or city == 'HBR Layout PC' or city == 'BLR':
        city = 'Bengaluru'
    elif city == 'Mumbai Hub' or city == 'BOM':
        city = 'Mumbai'
    elif city == 'Del':
        city = 'Delhi'
```

```
elif city == 'PNQ Pashan DPC' or city == 'PNQ Vadgaon Sheri DPC':
    city = 'Pune'
elif city == 'MAA':
    city = 'Chennai'
elif city == 'FBD':
    city = 'Faridabad'
elif city == 'CCU':
    city = 'Kolkata'
elif city == 'AMD':
    city = 'Ahmedabad'
elif city == 'FBD':
    city = 'Faridabad'
elif city == 'GGN':
    city = 'Gurgaon'
elif city == 'GZB':
    city = 'Ghaziabad'

return [city, place, code]
```

```
extracted_df = segment_df['source_data'].apply(lambda x: extract_city_place_code(x))
segment_df[['source_city','source_place','source_code']] = pd.DataFrame(extracted_df.tolist(), index= segment_df.index)
extracted_df = segment_df['destination_data'].apply(lambda x: extract_city_place_code(x))
segment_df[['destination_city','destination_place','destination_code']] = pd.DataFrame(extracted_df.tolist(), index= segment_df.index)
segment_df = segment_df.drop(columns=['source_name', 'source_data', 'destination_name', 'destination_data'])
segment_df.head()
```

index		segment_key	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	start_scan_to_end_scan	actual_distance_1
0	0	153671191949943656_IND487001AAB_IND487551AAA	trip-training	2018-09-12 00:25:19.499696	thanos::sroute:0ac760f3-96cb-4046-bfd0-8bc4678...	FTL	153671191949943656		86.0
1	1	153671191949943656_IND487551AAA_IND464668AAA	trip-training	2018-09-12 00:25:19.499696	thanos::sroute:0ac760f3-96cb-4046-bfd0-8bc4678...	FTL	153671191949943656		204.0
2	2	153671237597058150_IND785690AAB_IND785682AAA	trip-training	2018-09-12 00:32:55.970840	thanos::sroute:db0f8027-8ade-4411-9aff-b26adaa...	Carting	153671237597058150		252.0
3	3	153671262893947351_IND500055AAC_IND501401AAC	trip-training	2018-09-12 00:37:08.939733	thanos::sroute:beb73e7f-71ff-4501-bc17-191b4f3...	Carting	153671262893947351		59.0
4	4	153671262893947351_IND501401AAC_IND500010AAA	trip-training	2018-09-12 00:37:08.939733	thanos::sroute:beb73e7f-71ff-4501-bc17-191b4f3...	Carting	153671262893947351		210.0

5 rows x 24 columns

```
segment_df['trip_creation_year'] = segment_df['trip_creation_time'].dt.year
segment_df['trip_creation_month'] = segment_df['trip_creation_time'].dt.month
segment_df['trip_creation_day'] = segment_df['trip_creation_time'].dt.day
segment_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1640 entries, 0 to 1639
Data columns (total 27 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   index                                1640 non-null   int64
 1   segment_key                          1640 non-null   object
 2   data                                1640 non-null   category
 3   trip_creation_time                   1640 non-null   datetime64[ns]
 4   route_schedule_uuid                 1640 non-null   object
 5   route_type                          1640 non-null   category
 6   trip_uuid                           1640 non-null   object
 7   start_scan_to_end_scan               1640 non-null   float64
 8   actual_distance_to_destination       1640 non-null   float64
 9   actual_time                          1640 non-null   float64
10   osrm_time                           1640 non-null   float64
11   osrm_distance                       1640 non-null   float64
12   segment_actual_time_sum              1640 non-null   float64
13   segment_osrm_distance_sum            1640 non-null   float64
14   segment_osrm_time_sum                1640 non-null   float64
15   od_time_diff_hour                   1640 non-null   float64
16   source_state                        1640 non-null   object
17   destination_state                   1640 non-null   object
18   source_city                         1640 non-null   object
19   source_place                        1640 non-null   object
20   source_code                         1640 non-null   object
21   destination_city                    1640 non-null   object
22   destination_place                   1640 non-null   object
23   destination_code                    1640 non-null   object
24   trip_creation_year                  1640 non-null   int32
25   trip_creation_month                 1640 non-null   int32
26   trip_creation_day                   1640 non-null   int32
dtypes: category(2), datetime64[ns](1), float64(9), int32(3), int64(1), object(11)
memory usage: 304.7+ KB

```

▼ In-depth analysis

Grouping and aggregating at trip-level

Group the segment_df by trip_uuid

```

trip_dict = {
    'segment_key' : 'first',
    'data' : 'first',
    'trip_creation_time' : 'first',
    'route_schedule_uuid' : 'first',
    'route_type' : 'first',
    'start_scan_to_end_scan' : 'sum',
    'actual_distance_to_destination' : 'sum',
    'actual_time' : 'sum',
    'osrm_time' : 'sum',
    'osrm_distance' : 'sum',
    'segment_actual_time_sum' : 'sum',
    'segment_osrm_distance_sum' : 'sum',
    'segment_osrm_time_sum' : 'sum',
    'od_time_diff_hour' : 'sum',

```

```

    'source_state' : 'first',
    'destination_state' : 'last',
    'source_city' : 'first',
    'source_place' : 'first',
    'source_code' : 'first',
    'destination_city' : 'last',
    'destination_place' : 'last',
    'destination_code' : 'last',
}
trip_df = segment_df.groupby('trip_uuid').agg(trip_dict).reset_index()

```

```
trip_df.head()
```

	trip_uuid		segment_key	data	trip_creation_time	route_schedule_uuid	route_type	start_scan_to_end_scan	actual_distance_to_dest
0	trip-153671191949943656		trip-153671191949943656_IND487001AAB_IND487551AAA	training	2018-09-12 00:25:19.499696	thanos::sroute:0ac760f3-96cb-4046-bfd0-8bc4678...	FTL	290.0	9
1	trip-153671237597058150		trip-153671237597058150_IND785690AAB_IND785682AAA	training	2018-09-12 00:32:55.970840	thanos::sroute:db0f8027-8ade-4411-9aff-b26adaa...	Carting	252.0	3
2	trip-153671262893947351		trip-153671262893947351_IND500055AAC_IND501401AAC	training	2018-09-12 00:37:08.939733	thanos::sroute:beb73e7f-71ff-4501-bc17-191b4f3...	Carting	269.0	2
3	trip-153671320807895983		trip-153671320807895983_IND121004AAB_IND121102AAA	training	2018-09-12 00:46:48.079257	thanos::sroute:8c5ab716-198a-4395-b83f-5672773...	Carting	259.0	7
4	trip-153671549993828365		trip-153671549993828365_IND384205AAA_IND384355AAB	training	2018-09-12 01:24:59.938573	thanos::sroute:82facc11-0f66-496b-9d39-fa3891f...	FTL	562.0	17

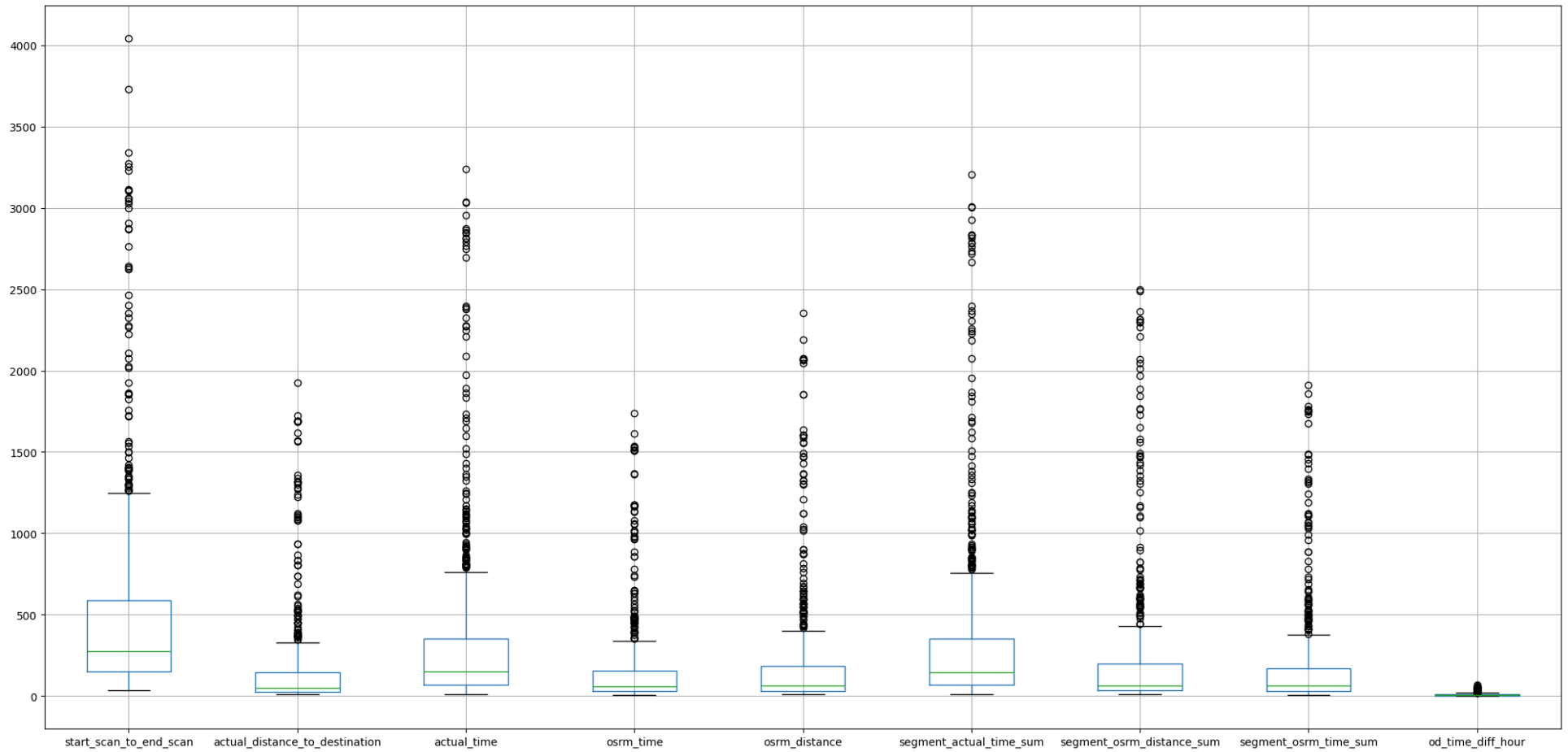
5 rows × 23 columns

▼ Outlier Detection & Treatment

```

trip_numerical_columns = ['start_scan_to_end_scan', 'actual_distance_to_destination',
                           'actual_time', 'osrm_time', 'osrm_distance', 'segment_actual_time_sum',
                           'segment_osrm_distance_sum', 'segment_osrm_time_sum', 'od_time_diff_hour']
trip_df[trip_numerical_columns].boxplot(figsize=(25,12))
plt.show()

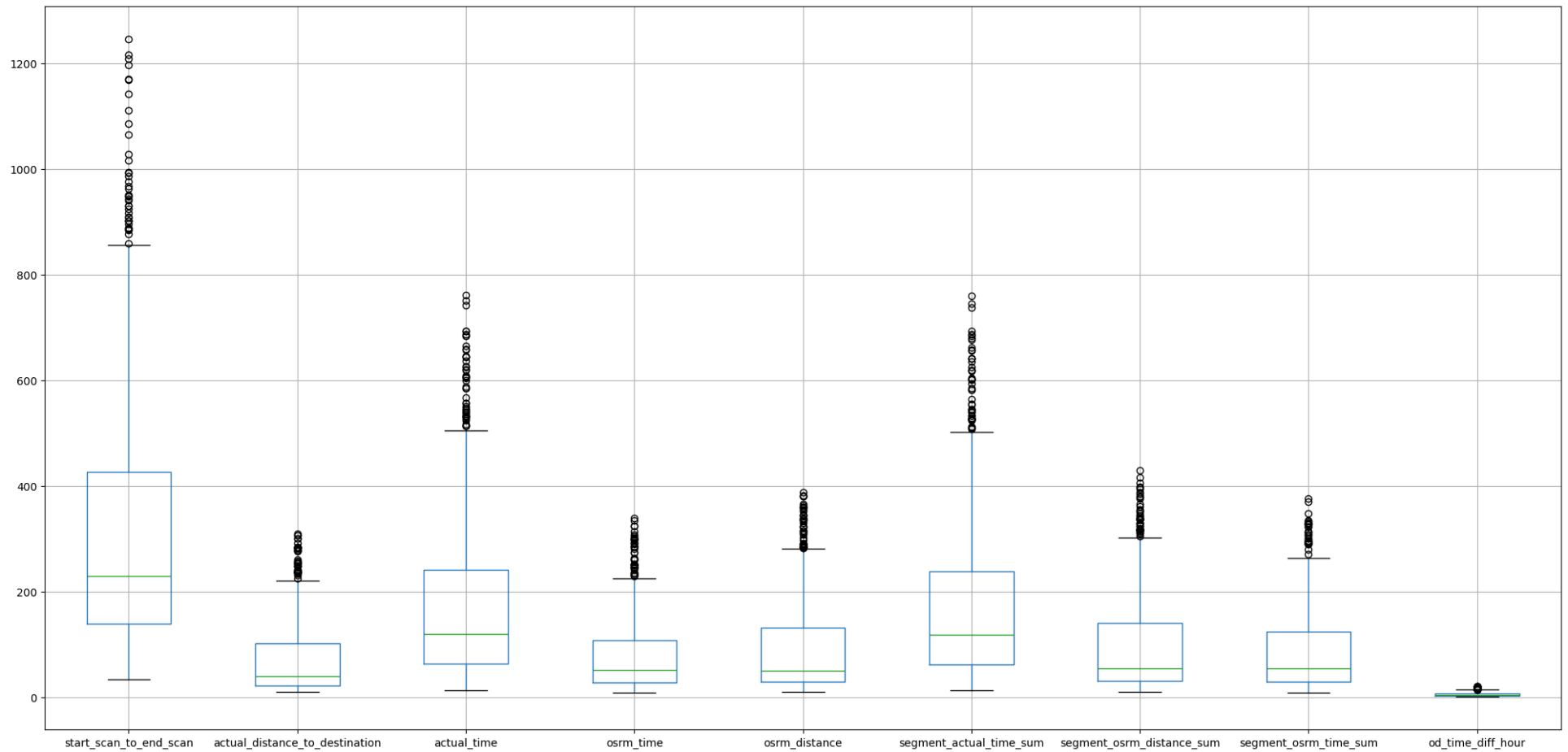
```



```
Q1 = trip_df[trip_numerical_columns].quantile(0.25)
Q3 = trip_df[trip_numerical_columns].quantile(0.75)
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5*IQR  
higher_bound = Q3 + 1.5*IQR  
trip_df = trip_df[~((trip_df[trip_numerical_columns] < lower_bound) | (trip_df[trip_numerical_columns] > higher_bound)).any(axis=1)]  
trip_df = trip_df.reset_index(drop=True)
```

```
trip_df[trip_numerical_columns].boxplot(figsize=(25,12))  
plt.show()
```



- ✓ Perform one-hot encoding on categorical features

route_type is the only categorical feature

```
ohe_df = pd.get_dummies(trip_df['route_type'], dtype='int', prefix='route_type')
trip_df = pd.concat([trip_df, ohe_df], axis=1)
trip_df = trip_df.drop(columns='route_type')
```

```
trip_df.head()
```

	trip_uuid	segment_key	data	trip_creation_time	route_schedule_uuid	start_scan_to_end_scan	actual_distance_to_destination	act
0	trip-153671191949943656	trip-153671191949943656_IND487001AAB_IND487551AAA	training	2018-09-12 00:25:19.499696	thanos::sroute:0ac760f3-96cb-4046-bfd0-8bc4678...	290.0	99.975595	
1	trip-153671237597058150	trip-153671237597058150_IND785690AAB_IND785682AAA	training	2018-09-12 00:32:55.970840	thanos::sroute:db0f8027-8ade-4411-9aff-b26adaa...	252.0	39.495954	
2	trip-153671262893947351	trip-153671262893947351_IND500055AAC_IND501401AAC	training	2018-09-12 00:37:08.939733	thanos::sroute:beb73e7f-71ff-4501-bc17-191b4f3...	269.0	24.359086	
3	trip-153671320807895983	trip-153671320807895983_IND121004AAB_IND121102AAA	training	2018-09-12 00:46:48.079257	thanos::sroute:8c5ab716-198a-4395-b83f-5672773...	259.0	76.231506	
4	trip-153671549993828365	trip-153671549993828365_IND384205AAA_IND384355AAB	training	2018-09-12 01:24:59.938573	thanos::sroute:82facc11-0f66-496b-9d39-fa3891f...	562.0	176.405153	

5 rows x 24 columns

```
print('Number of Carting route is ', trip_df[trip_df['route_type_Carting'] == 1]['route_type_Carting'].count())
print('Number of FTL route is ', trip_df[trip_df['route_type_FTL'] == 1]['route_type_FTL'].count())
```

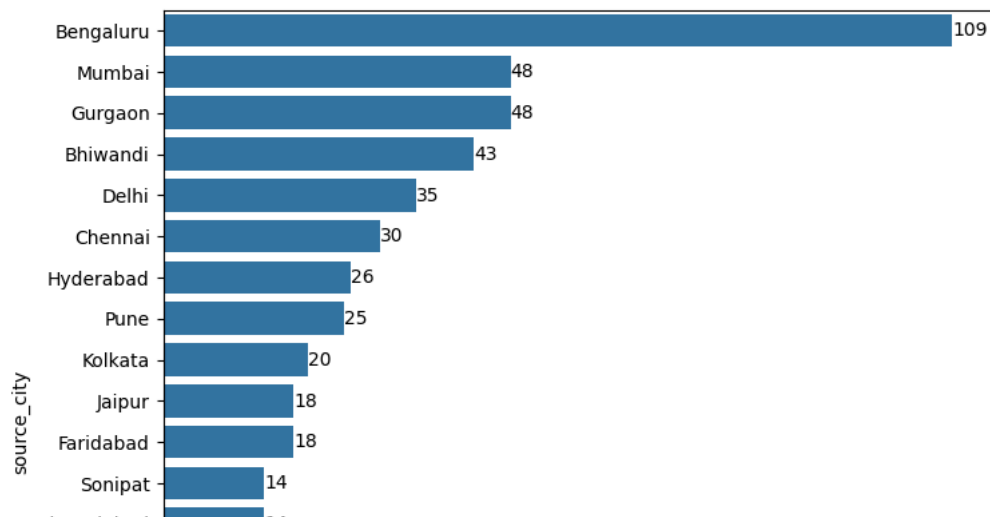
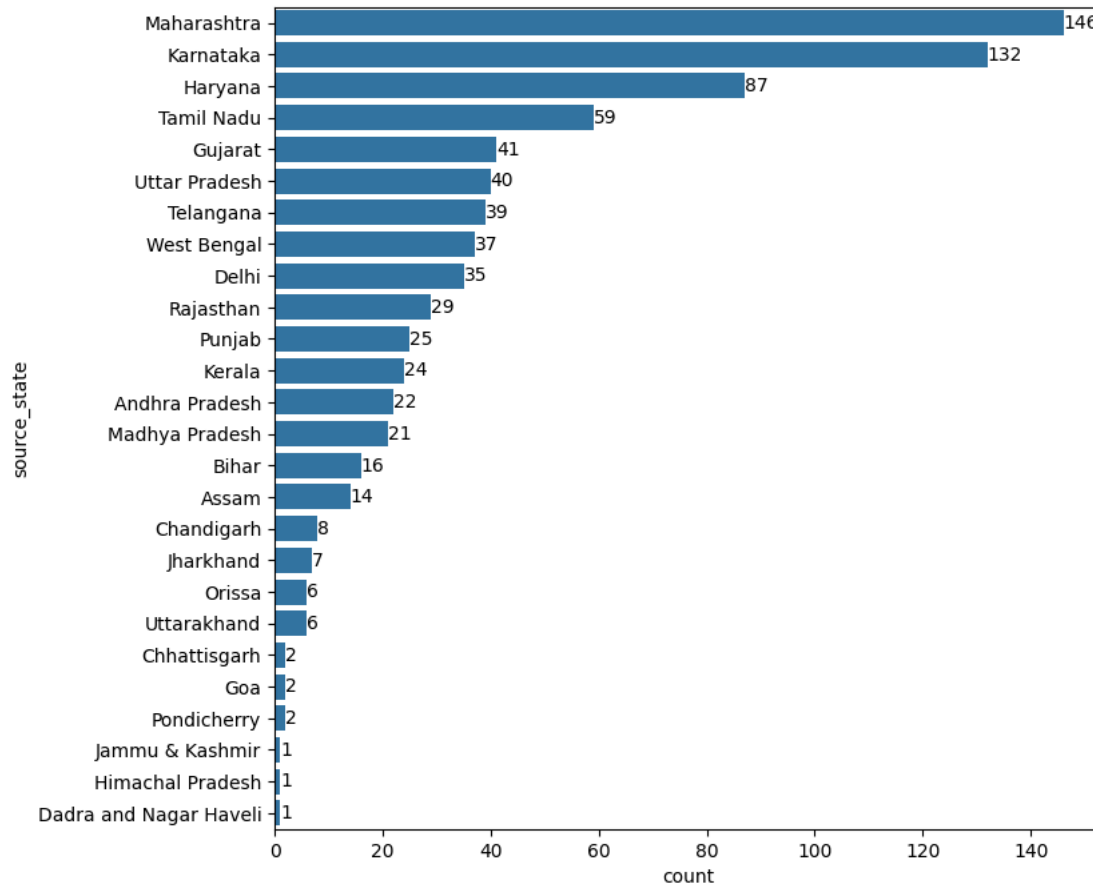
```
Number of Carting route is 560
Number of FTL route is 243
```

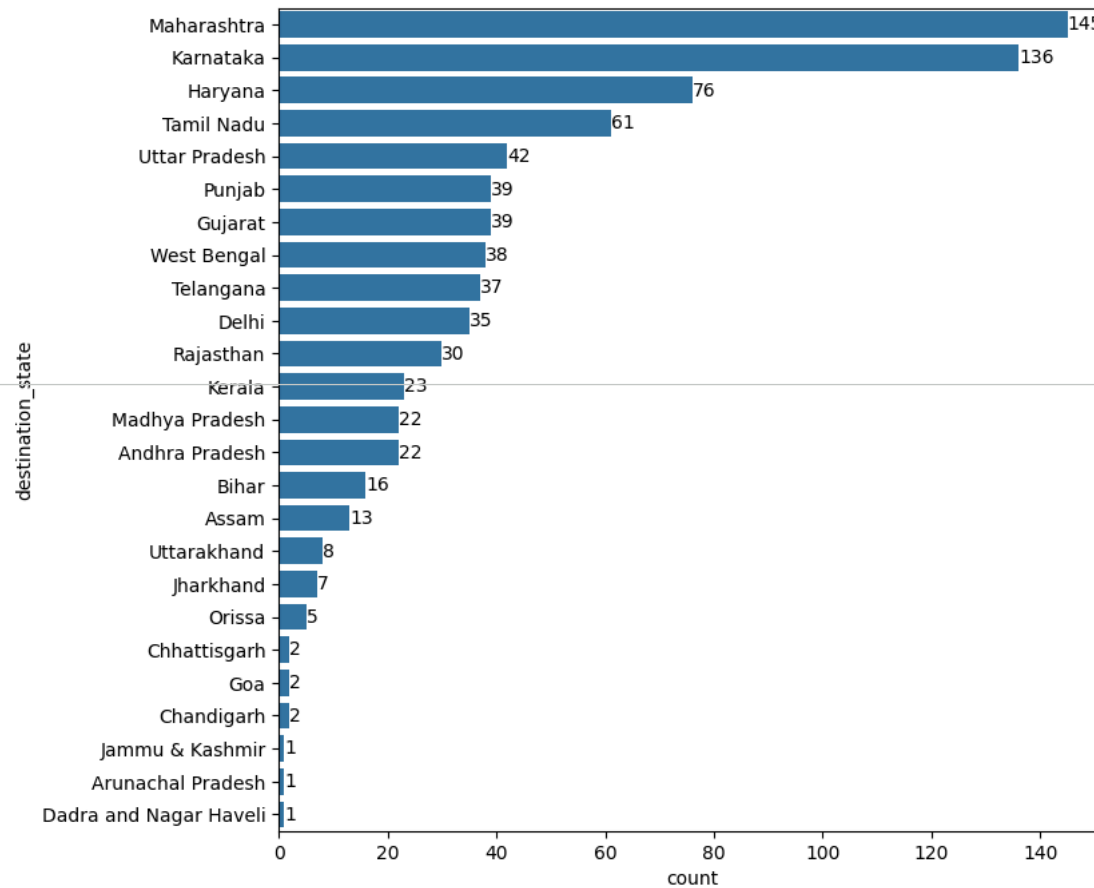
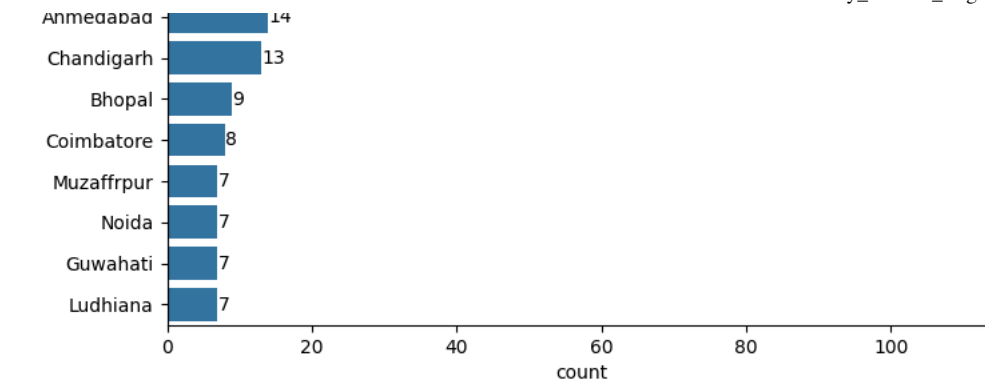
```
plt.figure(figsize=(8,8))
data = trip_df["source_state"]
ax=sns.countplot(y = data, order=data.value_counts().index)
ax.bar_label(ax.containers[0])
plt.show()

plt.figure(figsize=(8,8))
data = trip_df["source_city"]
ax=sns.countplot(y = data, order=data.value_counts()[:20].index)
ax.bar_label(ax.containers[0])
plt.show()

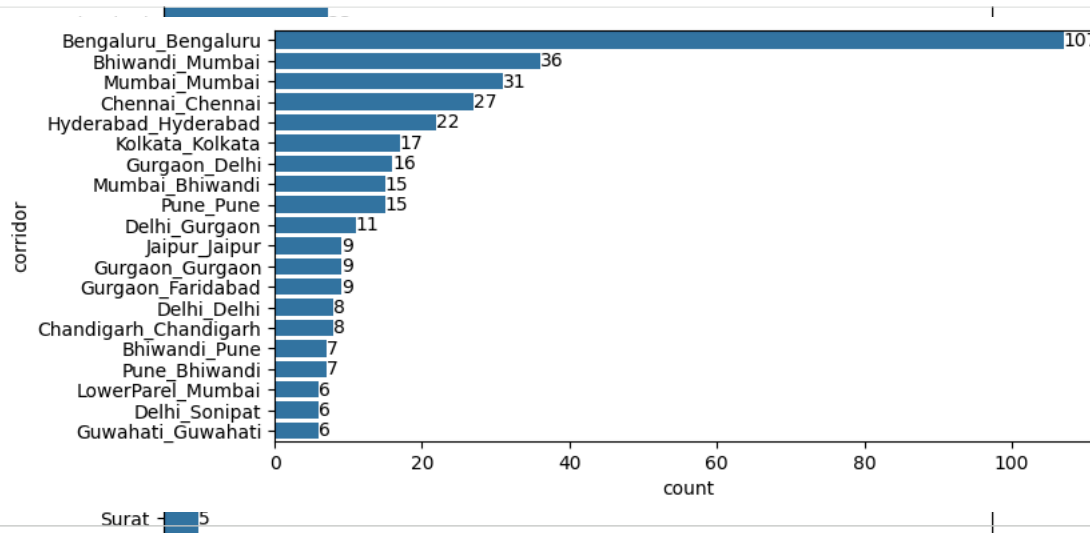
plt.figure(figsize=(8,8))
data = trip_df["destination_state"]
ax=sns.countplot(y = data, order=data.value_counts().index)
ax.bar_label(ax.containers[0])
plt.show()
```

```
plt.figure(figsize=(8,8))
data = trip_df["destination_city"]
ax=sns.countplot(y = data, order=data.value_counts()[ :20].index)
ax.bar_label(ax.containers[0])
plt.show()
```



```
trip_df["corridor"] = trip_df["source_city"] + '_' + trip_df["destination_city"]
plt.figure(figsize=(8,4))
ax=sns.countplot(y = trip_df["corridor"], order=trip_df["corridor"].value_counts()[:20].index)
ax.bar_label(ax.containers[0])
plt.show()
```

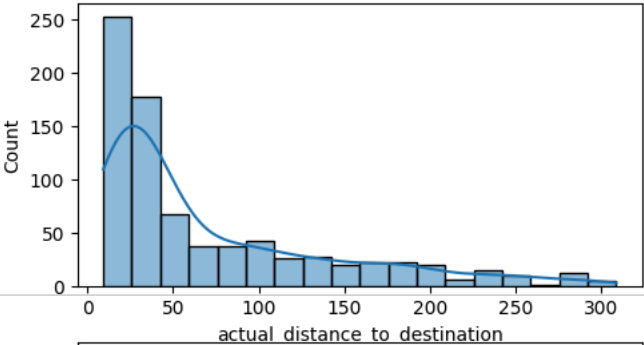
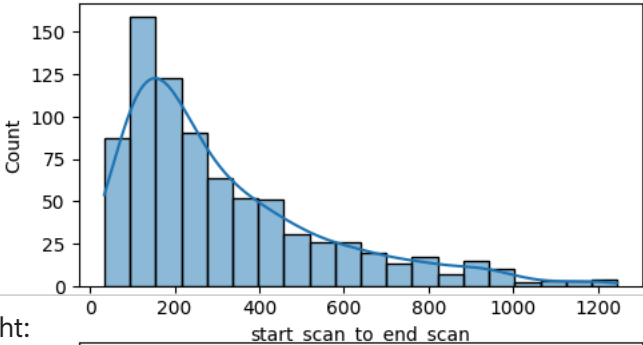


```
Mumbai_Bhiwandi_df = trip_df[((trip_df["corridor"] == "Bhiwandi_Mumbai") | (trip_df["corridor"] == "Mumbai_Bhiwandi"))]
print('Avg time: ', Mumbai_Bhiwandi_df['actual_time'].mean())
print('Avg distance: ', Mumbai_Bhiwandi_df['actual_distance_to_destination'].mean())
```

```
Avg time: 80.13725490196079
Avg distance: 21.923059279986813
```

▼ Normalize/ Standardize the numerical features using MinMaxScaler or StandardScaler

```
fig, ax = plt.subplots(nrows=5, ncols=2, figsize = (12, 16))
sns.histplot(data=trip_df, x = "start_scan_to_end_scan", kde=True, ax=ax[0,0])
sns.histplot(data=trip_df, x = "actual_distance_to_destination", kde=True, ax=ax[0,1])
sns.histplot(data=trip_df, x = "actual_time", kde=True, ax=ax[1,0])
sns.histplot(data=trip_df, x = "osrm_time", kde=True, ax=ax[1,1])
sns.histplot(data=trip_df, x = "osrm_distance", kde=True, ax=ax[2,0])
sns.histplot(data=trip_df, x = "segment_actual_time_sum", kde=True, ax=ax[2,1])
sns.histplot(data=trip_df, x = "segment_osrm_distance_sum", kde=True, ax=ax[3,0])
sns.histplot(data=trip_df, x = "segment_osrm_time_sum", kde=True, ax=ax[3,1])
sns.histplot(data=trip_df, x = "od_time_diff_hour", kde=True, ax=ax[4,0])
plt.show()
```

Insight:

None of the data is gaussian, so we will use MinMaxScaler

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(trip_df[trip_numerical_columns])
trip_df[trip_numerical_columns] = scaler.transform(trip_df[trip_numerical_columns])
```

trip_df.describe()

	trip_creation_time	start_scan_to_end_scan	actual_distance_to_destination	actual_time_osrm_time	osrm_distance	segment_actual_time_sum	segment_osrm_distance_sum
count	803	803.000000	803.000000	803.000000	803.000000	803.000000	803.000000
mean	2018-09-22 14:22:30.984476160	0.233833	0.209728	0.222163	0.212398	0.220562	0.208766
min	2018-09-12 00:25:19.499696	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	2018-09-17 00:31:12.432029696	0.085879	0.041962	0.066845	0.060423	0.065684	0.050090
50%	2018-09-22 05:03:15.439192064	0.161024	0.102395	0.143048	0.129909	0.140751	0.106279
75%	2018-09-27 22:30:54.411694592	0.323699	0.307281	0.304444	0.320103	0.302279	0.311008
max	2018-10-03 23:54:58.720536	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
std	NaN	0.200566	0.231793	0.208993	0.213279	0.208280	0.219477

Hypothesis Testing

Are aggregated actual time and aggregated osrm time similar?

- H0 : actual time and osrm time are similar
- H1 : actual time and osrm time are different